Feature Scaling

Beginner

Does Linear Regression actually need it?

Ordinary Linear Regression (OLS)	Gradient Descent / Regularized Regression
Scaling is not generally required for correct predictions — the closed-form solution handles different feature scales mathematically	Scaling matters a lot — features on very different scales can make gradient descent converge slowly, and regularization penalties are scale-sensitive

EXAMPLE

A house-price model with "area in square feet" (values in the thousands) and "number of bedrooms" (values 1–5) will still fit correctly with plain OLS. But feed those same unscaled features into Ridge or Lasso, and the penalty will unfairly shrink the bedroom coefficient much less than it should relative to area — so those methods typically standardize features first.

Categorical Variables

Intermediate

Regression only understands numbers — here's how text categories become numeric inputs.

ONE-HOT ENCODING

Each category becomes its own 0/1 column. For City ∈ {Mumbai, Delhi, Bangalore}, you'd get three columns: `is_Mumbai`, `is_Delhi`, `is_Bangalore` — each row has a 1 in exactly one of them.

City	is_Delhi	is_Bangalore
Mumbai	0	0
Delhi	1	0
Bangalore	0	1

THE DUMMY-VARIABLE TRAP

Notice only two columns were needed for three cities — Mumbai is implied when both are 0. Including a column for every single category *plus* an intercept creates perfect multicollinearity (the columns always sum to 1), which breaks the normal equation. Always drop one category as the reference/baseline.

Polynomial Regression

Intermediate

Fitting curves — while staying, mathematically, "linear regression."

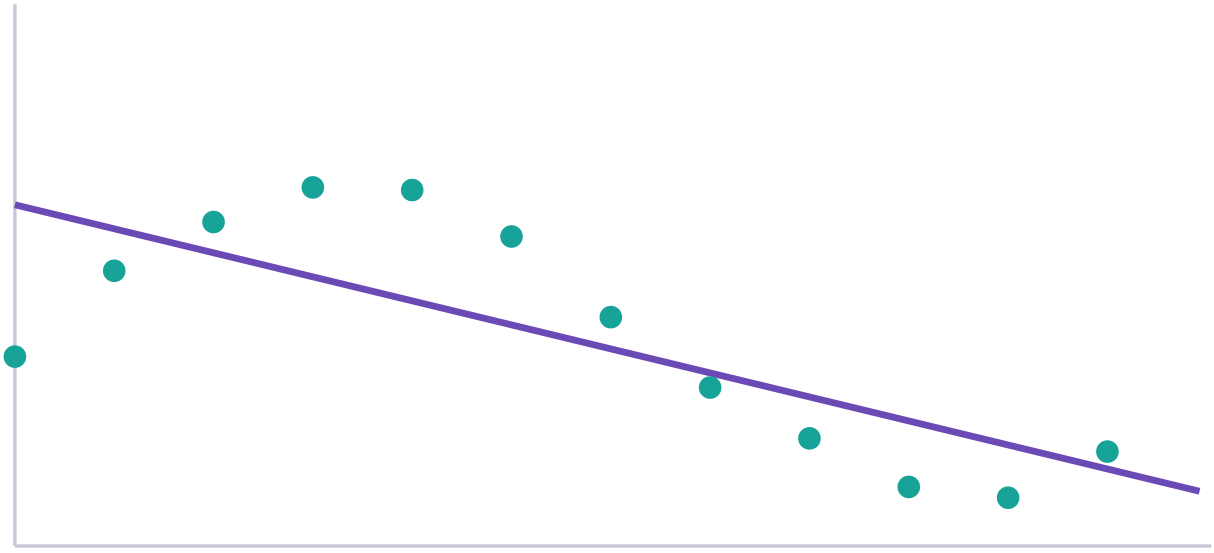
EXAMPLE

$$y = b_0 + b_1 x + b_2 x^2$$

WHY THIS STILL COUNTS AS "LINEAR" REGRESSION

Polynomial Regression is still linear *in the coefficients* — b_0, b_1, and b_2 each multiply a fixed input column (x, x^2, ...) additively, exactly like ordinary multiple regression, just with x^2 treated as if it were simply another feature. The curve in x-y space comes from the feature engineering, not from any change to how OLS solves for coefficients.

Interactive — Polynomial Fit & Model Complexity



Degree 1 — underfit: misses the curve entirely. Training SSE = 14.00

Same curved dataset, three model complexities. Degree 9 chases every training point, including noise — this is overfitting.

Regularization

Advanced

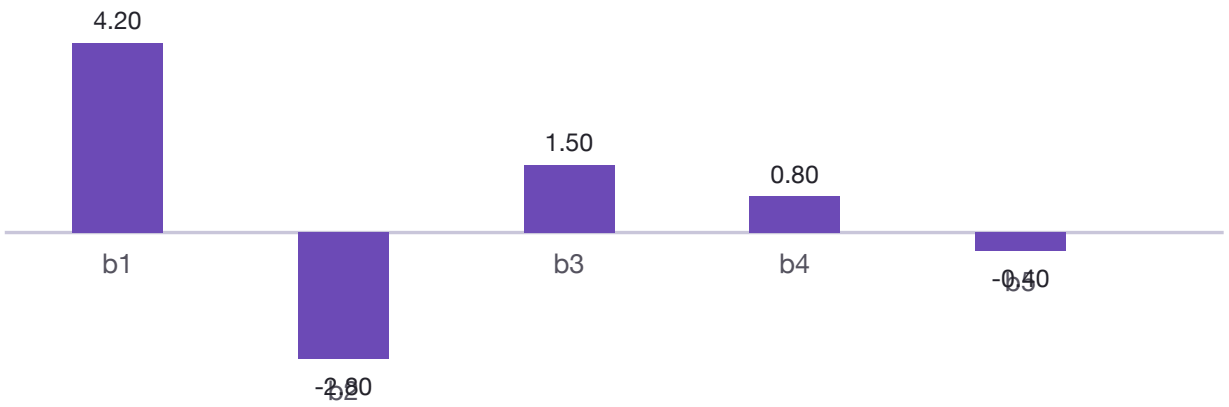
Ridge, Lasso, and Elastic Net — controlling complexity on purpose.

THE PROBLEM THIS SOLVES

What happens when we have too many features, or features that are highly correlated with each other? Coefficients can blow up to large, unstable values that fit the training data suspiciously well but generalize poorly. Regularization adds a penalty for coefficient size directly into the loss function being minimized.

Method	Penalty	Effect
Ridge	L2 (sum of squared coefficients)	Shrinks coefficients toward zero, rarely to exactly zero
Lasso	L1 (sum of absolute coefficients)	Can shrink some coefficients to exactly zero — performs feature selection
Elastic Net	Blend of L1 and L2	Combines shrinkage with occasional exact zeroing

Interactive — Ridge vs. Lasso Shrinkage



Ridge shrinks smoothly at λ=0 — none reach exactly zero.

Five example coefficients shrinking as λ increases. Watch how Lasso can drive small coefficients all the way to zero, while Ridge only ever shrinks them toward — not to — zero.

WHY LASSO PERFORMS FEATURE SELECTION

The L1 penalty's geometry (a diamond-shaped constraint region, versus Ridge's circular one) means the optimal solution is more likely to land exactly on an axis — where one or more coefficients are precisely zero — rather than merely close to it.

Bias and Variance

Intermediate

The trade-off underneath every modeling decision you'll ever make.

Underfitting	Good fit	Overfitting
High bias — model too simple to capture the pattern	Balanced — captures the real pattern, ignores noise	High variance — model captures noise as if it were pattern
Poor performance on train *and* test	Good performance on train and test	Great performance on train, poor on test

This directly connects back to the polynomial widget in Chapter 22 (model complexity / polynomial degree) and the regularization widget in Chapter 23 (adding a penalty to rein in an overfit model). As model complexity rises, training error keeps falling — but test error eventually turns upward. The best model sits at that turning point, not at either extreme.

CHAPTER 25

Complete ML Workflow

Beginner

Zooming all the way out — where everything in this course fits into a real project.

Raw Data → Understand business problem → EDA → Clean data → Select features → Train/test split → Preprocessing → Train model → Predict → Evaluate → Residual diagnostics → Tune / improve → Validate → Deploy / communicate

Every arrow here maps to a chapter you've already read: EDA and cleaning inform which assumptions might already be violated (Ch. 16); the split protects your evaluation (Ch. 12); training is OLS or gradient descent (Ch. 5, 9); evaluating means MAE/MSE/RMSE/R^2 (Ch. 13–14); diagnostics means residual analysis (Ch. 17); tuning often means regularization or feature changes (Ch. 18, 23).